

Attacking Recommender Systems with Agentic AI: An Adaptive, Role-Switching Group Shilling Attack

A step-by-step walkthrough, from first principles to the code and the figures

Abstract

This document explains the system in this code folder (`agent_attack_rs`) from the very beginning. It assumes you know nothing. The code is the engine behind the paper “*An Efficient and Effective Agentic Group Shilling Attack on Recommender Systems*” (AGAS). The compiled paper is `../main.pdf`.

The goal of the system is simple to state. We want to make one chosen item, the **target item**, show up in the recommendation lists of ordinary users. We do this by adding a small number of **fake users** to a recommender. What is new is *how* those fake users behave. A central **Coordinator**, which is a language-model agent, watches how the platform reacts. Every round it does two things. First it picks one **strategy**. Then it gives each fake user a **role** out of four. The four roles are Profiler, Sniper, Camoufleur, and Inactive. The roles change from round to round, and the strategy adapts when the platform pushes back. Because of this the fake activity never settles into the fixed pattern that detectors look for.

The document goes from the lowest level to the highest. We start with what a recommender and a shilling attack even are (Part I). We then look at the smallest possible action, one fake rating (Part II). We build up to one fake user with a role (Part III), then the signals the Coordinator reads (Part IV), then the Coordinator and its eight strategies (Part V), then the victim models and how a rank is measured (Part VI), then the round loop that ties it all together (Part VII), and then the full command-line pipeline (Part VIII). Part IX shows how the many output files turn into the single numbers printed in the paper. Part X is the reproduction guide. For every picture in `main.pdf` it says what the picture shows, which command makes the numbers, and which script draws it. Part XI collects every equation and every file in one place.

Every symbol is defined before it is used. Every idea is tied to the exact file and function that implements it. That way a new reader can trace “paper to code” without guessing.

Contents

Part I — The problem, stated from nothing	4
1 What a recommender system is	4
2 What a shilling attack is	4
3 How we measure success	4
4 The threat model and the objective	5

Part II — The lowest level: one fake rating	6
5 The atom of the attack: a RatingAction	6
6 How a rating enters the world	6
7 The poisoned matrix grows over time	6
Part III — One fake user and its four roles	7
8 A worker is a fake user with a policy	7
9 The four roles {PR, SN, CA, IN}	7
10 How a role becomes a prompt	7
Part IV — The signals the Coordinator reads	8
11 Worker signals: trust τ , risk γ , validator ϕ	8
12 Environment signals: $\rho, \Delta\rho, \eta, \xi, a$	9
Part V — The Coordinator and its eight strategies	10
13 What the Coordinator does each round	10
14 The eight strategies	10
15 The victim-probe state machine and the guardrail overlays	10
16 Complexity	11
Part VI — The victims, and how a rank is measured	11
17 The eleven victim models	11
18 Measuring the target rank	12
19 Bridge items, and why graph victims need them	12
Part VII — The round loop (Algorithm 1)	12
20 The episode runner	12
21 Two ways to score the attack: Option A and Option B	13

Part VIII — The end-to-end pipeline (how to run)	13
22 Install	13
23 Step 1 — get and preprocess the data	13
24 Step 2 — run one attack episode	13
25 Step 3 — the four reviewer scripts, one per research question	14
26 The output JSON, field by field	14
 Part IX — From many runs to one number	 14
27 The reporting convention	14
28 The aggregation helper	14
 Part X — Figure and table reproduction guide	 15
28.1 Honest status: what this repo generates, and what it does not	16
28.2 The one chain every quantitative figure shares	16
28.3 Performance: the main table and the radar (RQ1)	17
28.4 Stealthiness: the two t-SNE panels (RQ2)	18
28.5 Detector: the table and the Rec@10 bars (RQ3)	19
28.6 Ablation: two heatmaps and a table (RQ4)	19
28.7 Efficiency: rounds, time, tokens, and budget (RQ5)	22
28.8 Tables at a glance	23
 Part XI — Everything in one place	 24
29 Every equation, and where it lives	24
30 Every file that matters	24
31 The whole pipeline in one paragraph	24

Part I — The problem, stated from nothing

1 What a recommender system is

A recommender system watches what each user interacts with. It then tries to guess what else that user would like. Write $\mathcal{U} = \{1, \dots, M\}$ for the users and $\mathcal{I} = \{1, \dots, N\}$ for the items. The only thing we record is a big table of zeros and ones,

$$\mathbf{R} \in \{0, 1\}^{M \times N}, \quad R_{ui} = \begin{cases} 1 & \text{user } u \text{ interacted with item } i, \\ 0 & \text{otherwise.} \end{cases}$$

This is called **implicit feedback**. We do not need star ratings. We only need to know that an interaction happened. In this code the table is a pandas `DataFrame` with three columns, `user_id`, `item_id`, and `rating`. It is loaded and cleaned in `src/agas/cli.py::_load_processed_tables`.

The recommender learns a scoring function $\hat{y}_{ui} = f_{\Theta}(u, i)$. This function gives one real number for every pair of a user and an item. To make a recommendation, it takes the items a user has not seen yet, sorts them by that score, and returns the top ones:

$$\text{TopK}(u) = \arg \text{topK}_{q \in \mathcal{I} \setminus \mathcal{I}_u^+} \hat{y}_{uq}, \quad \mathcal{I}_u^+ = \{i : R_{ui} = 1\}. \quad (1)$$

Here $\mathcal{I}_u^+$ is the set of items that user u already interacted with. This is Eq. (topk_def) in the paper’s `formulation.tex`.

Remark 1 (Which f_{Θ} ?). The code has eleven concrete choices of f_{Θ} . They include matrix factorisation, neural collaborative filtering, and graph networks. They all live under `src/agas/recsys/targets/`. There is also a fast lightweight surrogate in `src/agas/recsys/surrogate.py` for the inner loop. We return to all of these in Part VI. For now, think of f_{Θ} as a black box that scores items.

2 What a shilling attack is

A **shilling attack** is also called profile injection, or poisoning. The idea is this. An attacker creates fake accounts and gives them fake interaction histories. The recommender then retrain on this polluted data. After retraining, one chosen *target item* i_{tar} is recommended to more real users than it deserves.

We can write this down. The attacker adds M_f fake users. Each fake user w has its own interaction vector $\tilde{\mathbf{r}}_w \in \{0, 1\}^N$. Stack these vectors into a matrix $\tilde{\mathbf{R}} \in \{0, 1\}^{M_f \times N}$. Then place that matrix below the clean table:

$$\mathbf{R}' = [\mathbf{R}; \tilde{\mathbf{R}}]. \quad (2)$$

This is Eq. (poison_append) in `formulation.tex`. In the code the “placing below” is just `pd.concat` followed by a `refit`. See `src/agas/recsys/surrogate.py::append_interactions` and `src/agas/recsys/targets/base.py::append_interactions`.

3 How we measure success

We measure success on a set of ordinary *test* users $\mathcal{U}_{\text{test}}$. These are real users, and the attack never touches them. The main question is a hit rate. Does the target item appear in a test user’s top- K

list?

$$\text{HR@}K(i_{\text{tar}}) = \frac{1}{|\mathcal{U}_{\text{test}}|} \sum_{u \in \mathcal{U}_{\text{test}}} \mathbb{I}[i_{\text{tar}} \in \text{TopK}(u)]. \quad (3)$$

There is also a rank-weighted version, $\text{NDCG@}K$. It gives more credit when the item lands near the very top of the list:

$$\text{NDCG@}K = \frac{1}{|\mathcal{U}_{\text{test}}|} \sum_{u \in \mathcal{U}_{\text{test}}} \frac{\mathbb{I}[i_{\text{tar}} \in \text{TopK}(u)]}{\log_2(\text{rank}(i_{\text{tar}} | u) + 1)}. \quad (4)$$

These two are Eqs. (hr_k) and (ndcg_k) in `experiment.tex`.

There is one more metric. It is used to check that the attack stays *clean*. It is $\text{Rec@}K$, the hit rate of the test users’ *own* held-out items:

$$\text{Rec@}K = \frac{1}{|\mathcal{U}_{\text{test}}|} \sum_{u \in \mathcal{U}_{\text{test}}} \mathbb{I}[i_u^{\text{test}} \in \text{TopK}(u)], \quad (5)$$

where i_u^{test} is the item we hid from user u for testing. This is Eq. (recK) in `experiment.tex`. The idea is easy. If $\text{Rec@}K$ barely drops after the attack, then the recommender still serves real users well. That means the attack did not visibly break the system.

Remark 2 (A single-item shortcut used inside the code). Equations (3) and (4) average over many test users. That is expensive to compute after every round. So inside the simulation the code uses a cheaper stand-in. It tracks the *mean rank of the target item* over a group of receptive real users. It then turns one rank into HR and NDCG with a short formula:

$$\text{HR@}K = \mathbb{I}[\text{rank} \leq K], \quad \text{NDCG@}K = \frac{\mathbb{I}[\text{rank} \leq K]}{\log_2(\text{rank} + 1)}. \quad (6)$$

This lives in `src/agas/cli.py::_rank_to_hr_ndcg`. Rank is the thing the Coordinator can actually watch round by round, so it is the internal signal. The HR and NDCG in the paper’s big tables are the full averages of (3) and (4). We come back to this point in Part IX because it matters when you reproduce the numbers.

4 The threat model and the objective

The attacker is **black-box**. It cannot see the victim’s parameters, its gradients, or its training code. It cannot see which users are test users, and it cannot see their held-out items. All it sees is the ranked lists returned to *its own* fake accounts. This is stated in `formulation.tex`. In the code it is enforced by `DefenseConfig(black_box_mode=True)` (Part VI), which hides the defender’s private alerts from what the Coordinator observes.

Now put the goal and the stealth need together. The attacker wants to solve

$$\max_{\{\tilde{\mathbf{r}}_w\}} \frac{1}{|\mathcal{U}_{\text{test}}|} \sum_{u \in \mathcal{U}_{\text{test}}} \mathbb{I}[i_{\text{tar}} \in \text{TopK}(u; \mathcal{A}(\mathbf{R}'))] \quad \text{s.t.} \quad \mathcal{S}(\tilde{\mathbf{R}}; \mathbf{R}) \leq \varepsilon. \quad (7)$$

Here $\mathcal{A}$ is the victim’s training procedure. The term $\mathcal{S}$ measures how strange the fake profiles look next to the real ones. The tolerance ε is never a fixed constant in the code. Instead it is made real by the signals of Part IV, which tell the Coordinator when to back off. This is the whole point of the paper. Older attacks solve (7) once, offline, with a fixed template. AGAS instead builds $\tilde{\mathbf{R}}$ step by step over T rounds, and it adapts as the platform reacts.

The whole system in one sentence. A Coordinator agent adds fake ratings round by round. It chooses a strategy and a role for each fake user from black-box feedback. The target item’s rank rises, and the fake profiles stay hard to tell apart from real ones.

Part II — The lowest level: one fake rating

5 The atom of the attack: a RatingAction

Everything the attack does is built from one small thing. One fake user rates one item. In the code this is a small dataclass, `src/agas/agents/messages.py::RatingAction`. It carries `agent_id`, which says which fake user acts. It carries `item_id`, which says which item. It carries `rating`, which is the value. And it carries a free-text `reason`, which is why the language model chose it.

6 How a rating enters the world

A proposed rating is not accepted right away. First it passes through the environment’s defense filter, `src/agas/simulation/environment.py::_apply_action_with_defense`. This filter models a realistic platform in three steps.

- It compares the rating to the item’s current **bias** b_i , which is just the item’s average rating. The gap is $d = |r - b_i|$.
- If the gap d is large and the fake user has low trust, the rating is *discounted*. Its influence weight w becomes less than one, and the effective rating becomes $b_i + w(r - b_i)$.
- If the platform is in “lockdown” and the user is not trusted, the rating may be *dropped* entirely, with some probability.

Only accepted, effective ratings are added to $\tilde{\mathbf{R}}$. The result of each action, whether it was accepted, discounted, or dropped, is stored in an `ActionOutcome`. These outcomes are the raw material for every signal in Part IV.

Remark 3 (Why a rating can hurt on a graph victim). On graph models such as LightGCN, every new edge raises the target’s *degree*. The degree normalisation $D^{-1/2}AD^{-1/2}$ then weakens all of the target’s existing edges. So a naive “rate the target 5.0” can actually *lower* its rank. This one fact is the reason the attack uses *bridge items* (Part VI) instead of rating the target directly on graph victims.

7 The poisoned matrix grows over time

Over the rounds $t = 0, \dots, T - 1$, the accepted actions pile up:

$$\tilde{\mathbf{R}} \leftarrow \tilde{\mathbf{R}} \cup \Delta \tilde{\mathbf{R}}^{(t+1)}, \quad \mathbf{R}^{(t)} = [\mathbf{R}; \tilde{\mathbf{R}}^{(\leq t)}].$$

After each round the victim is refit on $\mathbf{R}^{(t)}$, and a fresh target rank is read off. The final reported matrix is $\mathbf{R}' = [\mathbf{R}; \tilde{\mathbf{R}}]$ from (2). The piling up and the refit both happen in `environment.py::execute_step`, which calls `append_interactions(..., refit=True)`.

Part III — One fake user and its four roles

8 A worker is a fake user with a policy

Each fake user is a **worker** agent, `src/agas/agents/worker.py::WorkerAgent`. A worker holds a small mutable state, `WorkerState`, which stores its trust, its risk, how many actions it has taken, and its role history. Every round the Coordinator hands the worker a **role**. The worker then turns that role into a list of `RatingActions`. It does this by prompting a language model in `_act_with_llm`. If the model is not available, it falls back to simple deterministic rules.

9 The four roles {PR, SN, CA, IN}

The paper uses exactly four roles. The enum is `src/agas/roles.py::Role`. For the language-model protocol it is `messages.py::AgentRole`.

Symbol	Name	What it does, and which item pool it may touch
PR	Profiler	It makes a few safe filler ratings. It checks whether the platform still accepts ratings, and it helps find <i>bridge items</i> . Its pool is the popular or benchmark items, the cluster items, or the bridge pool.
SN	Sniper	This is the payload role. On embedding victims it rates the target directly. On graph victims it rates bridge or cluster items. It is given only to workers with high trust and low risk. Its pool is the target plus competitors, or the bridge pool.
CA	Camouflageur	This is the stealth role. It rates harmless, unrelated “noise” items so the fake user looks normal, and it rebuilds trust. Its pool is the noise items.
IN	Inactive	It does nothing this round. It is used to cool a worker down or to quarantine it.

The map from a role to its allowed item pool and its action budget is in `worker.py::_candidate_context`. The pools themselves are built by the environment in `environment.py::build_worker_context`. These pools are the benchmark, cluster, competitor, noise, bridge, flooding, and segment-profile pools. After the language model answers, the host *cleans* the reply in `worker.py::_sanitize_llm_actions`. Any item outside the allowed pool is thrown away, and every rating is clamped to the range `[1, 5]`. This makes sure a worker can only ever do what its role allows.

10 How a role becomes a prompt

Each role has a system prompt and a user prompt. They live under `prompts/worker_*/` and are loaded by `src/agas/llm/prompt_store.py::PromptStore`. If a file is missing, the code falls back to defaults in `worker.py::_default_prompt_text`. Those defaults are further tuned to the detected victim class, such as MF, LightGCN-small, LightGCN-large, or Sequential. The worker fills a small JSON context. That context holds its own signals, the allowed items, and a short memory of its own recent outcomes. It sends this to the model through `src/agas/llm/providers.py` and

reads back a strict-JSON list of actions. The number of tokens used by every call is added up for the efficiency figures.

Why role-switching gives stealth. A fake user that only ever snipes looks like a bot. A fake user that profiles, then camouflages, then snipes once, then goes quiet, looks like a person. The Coordinator changes roles every round, so no fake account ever settles into one fixed signature. This is the mechanism behind the stealth results in Part X.

Part IV — The signals the Coordinator reads

Every decision the Coordinator makes is driven by two families of numbers. Both live in `src/agas/signals.py` and match `method_coordinator.tex` in the paper. This is the mathematical heart of the system, so we build each number from scratch.

11 Worker signals: trust τ , risk γ , validator ϕ

For each worker w and round t we keep three numbers. Let $r_{t,w}$ be an accepted rating. Let b_i be the item bias. Let $d_{t,w} = |r_{t,w} - b_i|$ be the gap between them.

Trust $\tau_{t,w}$. Trust goes up by one only when the accepted rating looks benign. Benign means it stays within one point of the item’s own average:

$$\tau_{t+1,w} = \max(0, \tau_{t,w} + \mathbb{I}[d_{t,w} \leq 1]). \quad (8)$$

Risk $\gamma_{t,w}$. Risk goes up when the action is far from the bias. It goes up even more when the action is extreme and the worker has not yet earned trust:

$$\gamma_{t+1,w} = \max(0, \gamma_{t,w} + 0.5 \mathbb{I}[d_{t,w} > 1] + 0.5 \mathbb{I}[d_{t,w} > 1.5 \wedge \tau_{t,w} < 1]). \quad (9)$$

If instead the worker stays inactive that round, its risk cools down:

$$\gamma_{t+1,w} = \max(0, \gamma_{t,w} - 0.5). \quad (10)$$

An explicit alert or a block adds a further +0.5.

Validator $\phi_{t,w}$. Trust and risk track *recent actions*. The validator instead scores the *whole profile* for how suspicious it looks:

$$\phi_{t,w} = \min(1, 0.4 e_{t,w} + 0.3 h_{t,w} + 0.3 o_{t,w}). \quad (11)$$

Here e is the fraction of extreme ratings. The flag $h \in \{0,1\}$ is one when the worker gave a clear positive to the target. And o is the fraction of the worker’s items that at least two other fake users also rated, which is a group overlap. Equations (8) through (11) are implemented, one method each, in `signals.py::WorkerSignals`. The three methods are `update_after_action`, `apply_inactive_decay`, and `update_validator`. The profile score also has a standalone version in `src/agas/agents/profile_validator.py`.

12 Environment signals: $\rho, \Delta\rho, \eta, \xi, a$

These are round-level numbers. They describe how the *platform* is reacting.

Rank feedback $\rho^{(t)}$ and $\Delta\rho^{(t)}$. The number $\rho^{(t)}$ is the mean target rank, and lower is better. The change is $\Delta\rho^{(t)} = \rho^{(t-1)} - \rho^{(t)}$. A weak or negative $\Delta\rho$ tells the Coordinator that the current plan has stalled.

Acceptance rate η_t . This is the fraction of attempted actions that were accepted this round. A sudden drop hints that the platform is quietly filtering the attack.

Suppression signal $\xi_t = (q_t, s_t)$. This is the composite the Coordinator watches the most. Let $\mathcal{A}_t^{\text{try}}, \mathcal{A}_t^{\text{drop}}, \mathcal{A}_t^{\text{acc}}, \mathcal{A}_t^{\text{tar}}$ be the attempted, dropped, accepted, and target-related actions of round t . From these we build five normalised terms:

$$\begin{aligned} \hat{d}_t &= \frac{|\mathcal{A}_t^{\text{drop}}|}{\max(1, |\mathcal{A}_t^{\text{try}}|)} && \text{(dropped ratio),} \\ \hat{\delta}_t &= \min\left(1, \frac{1}{2|\mathcal{A}_t^{\text{acc}}|} \sum_a \delta(a)\right) && \text{(discount magnitude),} \\ \hat{m}_t &= \frac{1}{\max(1, |\mathcal{A}_t^{\text{tar}}|)} \sum_a \mathbb{I}[\Delta\rho^{\text{tar}}(a) < \epsilon_\rho] && \text{(weak target movement),} \\ \hat{s}_t &= \min(1, s_t/3) && \text{(normalised streak).} \end{aligned} \tag{12}$$

We also build a round-level group overlap. It is the largest pairwise Jaccard overlap between the workers' rated-item sets:

$$g_t = \max_{u \neq v} \frac{|A_{t,u} \cap A_{t,v}|}{|A_{t,u} \cup A_{t,v}|}. \tag{13}$$

The suspicion score gives all five terms equal weight:

$$q_t = \min(1, 0.2\hat{d}_t + 0.2\hat{\delta}_t + 0.2\hat{m}_t + 0.2\hat{s}_t + 0.2g_t). \tag{14}$$

The streak s_t goes up whenever the round shows filtering, discounting, weak movement, or no rank gain. Otherwise it resets to zero.

Alert flag $a_t \in \{0, 1\}$. This is a coarse trigger. It is set to one on an abnormal target spike, on very high overlap ($g_t \geq 0.8$), or on a sharp drop in acceptance.

All of (12) through (14), together with η_t and a_t , are computed in one method, `signals.py::EnvSignals.update`. The neutral start ($\eta_0 = 1$, $\xi_0 = (0, 0)$, $a_0 = 0$, $\Delta\rho^{(0)} = 0$) matches line 5 of Algorithm 1 in the paper and `signals.py::initial_env_signals`.

Remark 4 (Where the raw counts come from). The attacker never sees the defender's private state. So the counts that feed (12) are rebuilt from the black box. This is done by `src/agas/agents/defender.py::DefenseMonitorAgent.analyze`. It turns hidden accept, drop, and discount events into public "signals" that the Coordinator is allowed to read. That split is what keeps the threat model of (7) honest.

Part V — The Coordinator and its eight strategies

13 What the Coordinator does each round

The Coordinator is the brain, `src/agas/agents/coordinator.py::Coordinator`. Each round it does four things. First it reads the observation, which is all the signals of Part IV plus a short memory. Second it selects one round-level **strategy**. Third it gives each worker a role. Fourth, afterwards, it checks and gathers the accepted actions. The language model does the first three inside `LLMCoordinatorPolicy.assign`. After that, plain deterministic guardrails clean up the result, which we describe below. This whole thing is Algorithm 1, the outer loop, and Algorithm 2, the policy Π , in the paper.

14 The eight strategies

The enum is `src/agas/strategies.py::Strategy`. The plain descriptions are in `STRATEGY_DESCRIPTIONS` and in `method_strategies.tex`.

#	Name	Trigger and behaviour
S1	Victim Probe	In the early rounds, profilers probe to decide whether the victim is embedding-based or graph-based.
S2	Bridge Building	For graph victims only. Build a pool of bridge items for the snipers.
S3	Warm-up	Build trust τ with one profiler and a few camouflageurs before any sniper fires.
S4	First Push	The first coordinated payload. A few snipers act while a camouflageur stays on as cover.
S5	Silent Slowdown	Triggered when η_t drops but $a_t = 0$. Reduce pressure, and let high-trust workers act as camouflageurs.
S6	Profile Cleanup	Triggered when ϕ is high, or ξ is climbing, or $a_t = 1$. Freeze suspicious snipers and avoid overlap.
S7	Safe Replacement	In a high-risk state, rotate risky workers out and bring in rested, low- γ ones.
S8	Main Attack	In a calm state with the target still far, promote the best worker to sniper and keep pushing.

The language model emits a role for each worker. The code then *infers* which of S1 through S8 that pattern matches. This is done in `coordinator.py::_infer_strategy`, so the choice can be counted later for the ablation (Part X).

15 The victim-probe state machine and the guardrail overlays

Two parts of the Coordinator are pure code, not the language model.

Probe and classify (S1). Before the campaign, `Coordinator._probe_classify` runs a small state machine. It rates the target directly and watches $\Delta\rho$. If the direct signal helps, the victim is embedding-based, which the code calls MF-style. If it hurts or does nothing, the victim is graph-based, which the code calls LightGCN-style. An optional “sequential” probe tells apart recency-aware victims. The result sets `victim_model_class`, which then steers every prompt and item pool.

Overlays (Algorithm 2, part ii). After the language model proposes roles, a set of guardrails is applied in `Coordinator.assign_roles`. They stack on top of each other.

- **Suspicion lockdown**, in `_apply_lockouts`. A sniper that recently drew drops, discounts, or alerts is forced off the sniper role for a few steps.
- **Validator guardrail**, in `_apply_validator_guardrail`. A worker whose ϕ is above a threshold is forced INACTIVE.
- **Profiler presence**, in `_ensure_profiler_presence`. This guarantees a periodic scout probe even if the model forgot to add one.
- **Co-occurrence bridging**, in `_apply_cooccurrence_bridging`. This stamps the chosen bridge-item method into each assignment for graph victims.

These overlays are exactly the components the ablation study switches off (Part X), so each one has a stable name and a toggle.

16 Complexity

Let T be the number of rounds. Let $|\mathcal{U}_f|$ be the number of workers. Let L be the per-worker budget, and let V be the cost of one victim query. One full campaign then costs

$$O(T(c_C + |\mathcal{U}_f|(c_W + L) + V)).$$

In words, there are $O(T)$ heavy Coordinator calls and $O(T|\mathcal{U}_f|)$ short worker calls. This is the complexity section of `method.tex`. The key point for the efficiency figures is that the cost grows linearly with the rounds and the workers. There is no GAN, no bi-level solve, and no per-target surrogate to retrain.

Part VI — The victims, and how a rank is measured

17 The eleven victim models

The victims live in `src/agas/recsys/targets/`. They share a base class, `base.py::BaseTargetRecommender`. The base class handles the data indexing, implicit-versus-explicit training, `rank_item`, and `mean_scores_for_segment`. The concrete families are:

- **Matrix factorisation**, in `mf.py` (MF, GMF, BPR-MF, PMF, WMF, NMF), `neumf.py` (NeuMF and NCF), and `svdpp.py`.

- **Graph**, in `lightgcn.py` (LightGCN), `ngcf.py` (NGCF), and `gcmc.py`.
- **Others**, in `autorec.py` (autoencoders), `itemknn.py`, and `sequential.py`.

The map from a name to a class is `cli.py::_build_target_model`. There is also a fast CPU surrogate, `recsys/surrogate.py`, used for the inner loop when a full torch victim would be too slow.

18 Measuring the target rank

We give the model a set of segment users and a set of candidate items. Then `BaseTargetRecommender.rank_item` scores every candidate, sorts them, and returns the 1-based position of the target. The candidate set is the target’s genre or keyword cluster, from `environment.py::_resolve_target_cluster_items`. In transfer mode it can be all items. This single number is the $\rho^{(t)}$ of Part IV.

19 Bridge items, and why graph victims need them

We saw in Part II that rating the target directly can hurt on a graph victim. So the snipers rate *bridge items* instead. A bridge item sits on a short graph path from real target-fans to the target. Rating it lets graph diffusion carry the signal to the target, without inflating the target’s own degree. The selection methods are in `environment.py::compute_bridge_items`. They include co-occurrence, gradient, two-hop, low-degree and high-degree structural, and an “auto” method that chooses based on how many ratings the target already has. This is strategy S2.

Part VII — The round loop (Algorithm 1)

20 The episode runner

Everything above is tied together by `src/agas/simulation/episode.py::AGASEpisodeRunner.run`. This function *is* Algorithm 1 of the paper. For each round t it does the following.

1. It builds the observation $\mathbf{o}_t$ from the current signals and the short memory, in `environment.observation`.
2. It optionally scores every worker’s profile validator ϕ .
3. It calls `coordinator.assign_roles`. This returns one strategy and a role for each worker.
4. Each worker acts, in `worker.act`. This returns a batch of `RatingActions`.
5. It calls `environment.execute_step`. This applies the defense filter, updates τ and γ , adds the accepted rows, refits the victim, and reads the new rank.
6. It appends everything to `history`, and it stops early if the goal rank is reached.

At the very end it also computes victim-side embedding-cluster metrics, in `src/agas/metrics/embedding_cluster.py`, for the stealth analysis. These metrics are never shown to the attacker.

21 Two ways to score the attack: Option A and Option B

The transfer command, `cli.py::cmd_run_transfer`, supports two protocols.

- **Option A, offline transfer.** Run the whole campaign on an *episode* model, which is often the fast surrogate or a LightGCN. Then pull out the accepted fake interactions, add them to the clean data, and train *fresh* victim models from scratch. Finally compare the target’s rank on the clean data against the poisoned data.
- **Option B, in-loop.** Here the victim *is* the episode model, and it retrains every round. So the Coordinator gets true rank feedback. After the run, a “best-sequence retrain” keeps only rounds 0 through t^* , where $t^* = \arg \min_t \rho^{(t)}$, and retrains once more. This is the t^* protocol noted in the README.

Both protocols write per-victim fields into the output JSON. Those fields are `initial_rank`, `final_rank`, `best_rank`, `hr_at_k`, and `ndcg_at_k`.

Part VIII — The end-to-end pipeline (how to run)

This part is the practical “what do I type”. Every command is run from the repo root, `agent_attack_rs/`.

22 Install

```
pip install -e . # core AGAS loop (CPU only)
pip install -e '.[targets]' # add torch victims (LightGCN, NeuMF, ...)
export OPENAI_API_KEY=sk-... # required for the LLM Coordinator/workers
export OPENAI_MODEL=gpt-5.1 # optional, default model id
```

If `OPENAI_API_KEY` is not set, the language-model calls fail and the workers fall back to deterministic behaviour. That is fine for a dry run, but the paper numbers use the language model.

23 Step 1 — get and preprocess the data

Drop each raw dataset into `data/<name>/`. The links are in `README.md`. Then normalise everything into canonical `interactions.csv` and `items.csv` files:

```
python scripts/preprocess_all.py --data-root data --output-root processed
```

This calls `cli.py::cmd_preprocess`, which calls `src/agas/data/pipeline.py`. The per-dataset loaders are in `src/agas/data/loaders/`. The output lands in `processed/<name>/`. For a quick smoke test, the tiny bundled `ml-latest-small` sample needs no download.

24 Step 2 — run one attack episode

The single entry point is `scripts/run_agas.py`. It is a thin wrapper over `agas.clirun-episode`:

```
python scripts/run_agas.py \
  --dataset ml-100k --victim lightgcn \
  --rounds 18 --seed 42 --n_workers 8 --budget 0.01 \
  --out outputs/agas_ml100k_lightgcn_seed42.json
```

The seven flags map onto the paper’s protocol. Here T is `--rounds`, $|\mathcal{U}_f|$ is `--n_workers`, and L is `--budget`. The run writes a JSON trace, and next to it a human-readable `.log`. The log holds the per-round ranks, the strategies, the roles, the token usage, and the role and strategy activation tallies.

25 Step 3 — the four reviewer scripts, one per research question

The `bash/` folder runs a tiny sweep for each research question. Each script prints which figure or table it maps to.

```
bash bash/run_performance.sh      # RQ1 -> tables/benchmark_unpopular.tex + radar
bash bash/run_stealth_and_detect.sh # RQ2 + RQ3 -> t-SNE figs + detection table
bash bash/run_ablation.sh         # RQ4 -> ablation heatmaps + backbone table
bash bash/run_efficiency.sh        # RQ5 -> efficiency figures
```

For the full paper, widen the loops inside each script to all 6 datasets, 11 victims, and 5 seeds. The script `run_stealth_and_detect.sh` also runs the detector battery through `agas.clievaluate-detectors`.

26 The output JSON, field by field

A `run-episode` output carries several fields. It has `initial_rank`, `final_rank`, and `best_rank` with its step. It has `history`, which is the per-round observation, assignments, reports, and feedback. It has `token_usage`, which is the coordinator, per-agent, and aggregate token counts. It has `activation_stats`, which is the role and S1 through S8 strategy counts. And it has `embedding_cluster_metrics`. A `run-transfer` output instead carries `option_a` and `option_b`. Each of those is a dict keyed by victim with rank, HR, and NDCG before and after. These are exactly the fields Part IX reads.

Part IX — From many runs to one number

27 The reporting convention

The paper reports the **mean and the 95% confidence interval over 5 runs**. The 5 runs are 5 random seeds. In this guide we use seeds **42, 43, 44, 45, 46**. Every promotion metric is also scaled by 10^3 . So a table cell that reads “ 40.0 ± 0.2 ” means $\text{HR@10} = 0.0400 \pm 0.0002$. In other words, each cell is the aggregate of 5 JSON files that differ only in the `--seed` flag.

28 The aggregation helper

The AGAS engine writes one JSON per run. On its own it does not produce the aggregated cells. That last mile, from runs to numbers, is what `scripts/aggregate_results.py` does. This script was added alongside this document. It works in five steps.

1. It walks an output directory and loads every `*.json` inside it.
2. It recognises both schemas, the `run-episode` one and the `run-transfer` one.
3. It turns each reported rank into $\text{HR}@K$ and $\text{NDCG}@K$ with the same formula (6) the CLI uses, which is `_rank_to_hr_ndcg`.
4. It groups the runs by `(config,dataset,victim)` and prints the mean and the 95% CI, in the $\times 10^3$ convention.
5. It can also write a tidy CSV that a plotting script can read.

```
# aggregate a seed sweep into HR@10 / NDCG@10 / rank-delta with 95% CI
python scripts/aggregate_results.py outputs/multiseed_welltrained
# RQ1 sweep, also dump a CSV for the plot scripts
python scripts/aggregate_results.py outputs/rq1_performance --k 10 --csv summary.csv
# efficiency: also report mean tokens per run
python scripts/aggregate_results.py outputs/rq5_efficiency --tokens
```

Remark 5 (An honest caveat: proxy versus population HR). The released smoke outputs use the single-target-rank proxy (6). So their $\text{HR}@10$ is either 0 or 1 per run. It is not the smooth population average (3) used in the big tables. For this reason the aggregation script also reports `rank_delta`, which is how far the target climbed. That number is stable and always defined. To reproduce the population $\text{HR}@K$ exactly, you must evaluate (3) over the full $\mathcal{U}_{\text{test}}$ set on the retrained victim. The needed hooks, `rank_item`, `mean_scores_for_segment`, and the segment users, are all present in `base.py` and `environment.py`.

Part X — Figure and table reproduction guide

This is the part to read when the question is simply “which file do I run to get picture X ?”. Two facts frame everything.

- (1) **The drawing scripts only draw.** They live in `../code/`, the paper folder, and they are pure `matplotlib`. Given numbers, they render a `.png` into `../figures/`. Several of them still keep their numbers in a hard-coded array.
- (2) **The numbers come from this repo, `agent_attack_rs`.** You run the engine (Part VIII), then aggregate (Part IX), then paste the aggregated values into the drawing script’s data block, or point the script at the CSV.

The order to run things is always the same:

preprocess $\rightarrow$ bash RQ script $\rightarrow$ `aggregate_results.py` $\rightarrow$ drawing script in `../code/`.

Suggested first run

Start with the smallest end-to-end path, just to see the whole loop work:

```
python scripts/preprocess_all.py --data-root data --output-root processed
bash bash/run_performance.sh
python scripts/aggregate_results.py outputs/rq1_performance --csv summary.csv
python ../code/benchmark_popularity_regime_summary.py # renders the radar
```

28.1 Honest status: what this repo generates, and what it does not

Not every figure can be produced end to end by this repo alone, and it is only fair to say so plainly. This repo implements the **AGAS** attack. It does *not* re-implement the ten baseline attacks (RandomAttack, Bandwagon, AUSH, PoisonRec, PGA, GSPAttack, TargetedAttack, CLear, AgentSA, AgentAttack) or the five detectors (BaseDetect, DHAGCN, PCASelectUsers, GAGE, MD-CBA). So the AGAS row or column of every figure is generatable from the engine. The baseline and detector entries are not, and this document does not fabricate them. They must come from those methods’ own code.

Figure or table	Status in this repo
Main table (<code>benchmark_unpopular.tex</code>)	AGAS rows: yes. Baseline rows: not implemented.
Popularity / intro radar	AGAS polygon: yes. Baseline polygons: not implemented.
Budget trade-off	Yes, fully (AGAS only figure).
Ablation strategy heatmap	Yes, fully (added the <code>--disable-strategies</code> flag).
Ablation component / size heatmap	Yes, fully (roles, memory, signals, validator, coordinator flags).
Efficiency rounds trade-off	HR@10 per round: yes. Time line: yes, now that runtime is recorded.
Efficiency method vs time	AGAS bar and line: yes. Baseline bars and lines: not implemented.
Tokens-until-Top-10	AGAS tokens and seconds: yes. Baseline bars: not implemented.
Detector table (<code>detection_mf.tex</code>)	Not generatable: needs the 5 detectors and an <code>evaluate-detectors</code> command, neither present.
Benign Rec@10 bars	Not generatable: needs Rec@ K , which needs a benign train/test split the pipeline does not create.
Stealth t-SNE (both panels)	Embeddings exist, but no export script yet. The drawing scripts currently simulate the points.

Three of these gaps are quick and I closed the first: (1) the efficiency *time* axes needed wall-clock timing, so the episode now records `runtime_sec` (total) and `elapsed_sec/cumulative_sec` per round, read with `aggregate_results.py--timing`. (2) Rec@ K and (3) the detector battery are larger, because they need a benign split and the detector implementations respectively, so they remain honest gaps rather than stubs. If you want any specific baseline or detector implemented, that is a separate, scoped piece of work.

28.2 The one chain every quantitative figure shares

Before we go figure by figure, here is the one chain that turns a single run into a single plotted number. Every line, bar, and heatmap figure is just this chain, evaluated at different settings and then arranged on axes. It has four steps.

Step 1: a rank is produced. During a run, the victim is refit and the target’s position among the candidate items is read off. This is done by `src/agas/recsys/targets/base.py::rank_item` (line 132). That function scores every candidate, runs `torch.argsort(scores,descending=`

True), and returns the 1-based position of the target i_{tar} . Inside the loop it is called through `environment.py::_target_rank` (line 277) and stored as `self.current_rank` after every round, in `environment.py::execute_step` (line 706). Call this integer ρ .

Step 2: the rank becomes HR and NDCG. The CLI turns a rank into the two promotion metrics with the formula (6). This is `cli.py::_rank_to_hr_ndcg` (line 124), where $\text{HR}@K = \mathbb{I}[\rho \leq K]$ and $\text{NDCG}@K = \mathbb{I}[\rho \leq K] / \log_2(\rho + 1)$. In `cmd_run_transfer` this happens in the `forkinmetrics_ks` loop, at lines 1321–1326 for Option A and at lines 1462 and 1576 for the other paths. That loop writes `hr_at_k` and `ndcg_at_k`, together with the clean-versus-poisoned `initial_rank` and `final_rank` (lines 1307–1314), into the run’s JSON.

Step 3: the runs are averaged. We run each setting 5 times, once per seed (42, 43, 44, 45, 46), so there are 5 JSON files. The script `scripts/aggregate_results.py` reads those files. It re-applies the same `rank_to_hr_ndcg` (line 44), groups the runs by `(config, dataset, victim)` (line 198), and returns the mean and the 95% CI over the 5 seeds through `mean_ci` (line 51), scaled by 10^3 . Call the mean $\bar{y}$ and the CI half-width c .

Step 4: the number is placed on an axis. The mean $\bar{y}$ becomes a bar height, or a line’s y , or a heatmap cell value. The half-width c becomes the error bar. Which axis or cell it lands in depends only on the loop variable that produced the run. That variable is the round t , the budget b , the ablated component, the dataset, or the victim.

So for every figure below we only need three things. We need to know which loop variable is swept. We need the command that runs it. And we need to know how the resulting pair of a setting and its $\bar{y}$ is drawn. The efficiency figures replace Step 2 with token and time counters. The tokens come from `cli.py::_collect_token_usage` (line 615), in the field `token_usage.aggregate.total_tokens`. The role and strategy tallies come from `_compute_activation_stats` (line 661). The wall-clock comes from the run timing.

28.3 Performance: the main table and the radar (RQ1)

Main table benchmark_unpopular.tex (Tab. I)

The loop variable swept is the triple (dataset, victim, seed). Run it like this:

```
# widen the loops in run_performance.sh to all datasets/victims, 5 seeds
bash bash/run_performance.sh
python scripts/aggregate_results.py outputs/rq1_performance --k 10 --csv rq1.csv
```

Each printed line of `rq1.csv` is one cell block of the table. The column `hr@10_x1000_mean` and its `hr@10_x1000_ci` form the “ 40.0 ± 0.2 ” entry. The column `ndcg@10_x1000_mean` gives the NDCG entry. Steps 1 through 3 of the chain produce each number. The table just lays out the victim and the metric in columns, and the method in rows. Here “method” means which config produced the run. AGAS is the full system. The baselines are separate scripts, but their rows are filled the same way, going from rank to HR and NDCG to the mean and CI.

Popularity radar benchmark_popularity_regime_summary.py

This figure shows three victim families, which are MF, graph-conv, and graph-contrastive. In each family, every attack method is drawn as a filled quadrilateral over four axes. The four axes are NDCG@10, HR@10, NDCG@50, and HR@50.

The numbers come from RQ1 with head and mid targets across the three families. After aggregation you get, for each method m and family f , one vector $v_{m,f} = [\text{NDCG@10}, \text{HR@10}, \text{NDCG@50}, \text{HR@50}]$.

Now the drawing. In `benchmark_popularity_regime_summary.py` that vector is the dict entry `RAW[m][f]`. For example, `RAW["AGAS"]["mf"]` is `[58, 70, 70, 78]`. The four axes are angles $\theta_k = 2\pi k/4$ from `radar_angles`, with `set_theta_offset(pi/2)` and direction `-1`. For one method, vertex k of its polygon is the polar point $(\theta_k, v_{m,f}[k])$. The call `ax.plot(angles, values)` draws the outline, and `ax.fill` shades it, both inside `draw_radar`. So on the radar the angle tells you which metric you are on, and the radius tells you the value of that metric. To make the figure data-driven, replace the `RAW` block with your aggregate. The intro radar, `intro_fig_radar_motivation.py`, is built the same way, but it is split by popularity regime instead of by victim family.

28.4 Stealthiness: the two t-SNE panels (RQ2)

Left panel (`benchmark_fig7_tsne_distance_users_target.py`)

This panel has one small plot per method, six in all. It samples nine users. Next to each user it prints two numbers. The red number is the distance from that user to the target item. The black number is the user's average distance to its own ground-truth items. A method is best when both numbers are small. That means the target was pulled close to the user, and the user was not dragged away from its real tastes.

These two numbers come from a poisoned victim's learned vectors. After a run, the surrogate exposes `user_factors` and `item_factors` in `recsys/surrogate.py`. Embed the target item i_{tar} , each sampled user u , and the items u rated. Call the red number A_u and the black number B_u . Then $A_u = \|e_u - e_{i_{\text{tar}}}\|$ and $B_u = \frac{1}{|\mathcal{I}_u^+|} \sum_{i \in \mathcal{I}_u^+} \|e_u - e_i\|$. The way to reach the victim's vectors mirrors `src/agas/metrics/embedding_cluster.py`, which already pulls the factor matrices. That file uses cosine distance, and here we use Euclidean, but the hook is the same.

Now an important caveat about the drawing. In this script the *positions* of the stars are not the distances. They are a stylised layout, set by `ANGLES_DEG` and the per-method `LAYOUT[radius]`. The layout is the same across panels so the eye can compare spread. The real data are only the printed numbers. They live in the dict entry `PROFILES[method][user]`, which is the pair (A_u, B_u) . They are drawn by `ax.text(...)` at the star for the black number and at the connector midpoint for the red number, inside `plot_panel`. So to make it real, replace the `PROFILES` block with your measured (A_u, B_u) . The layout stays decorative on purpose.

Right panel (`dappoison_fig6_tsne.py`)

This panel has one 2-D t-SNE scatter per method. Blue dots are clean users, and red stars are injected fake users. Diffuse red is stealthy, and clustered red is easy to detect.

The numbers come from a poisoned matrix $\mathbf{R}'$. Get each user's history embedding from the victim, then run `sklearn.manifold.TSNE` down to two dimensions. The clean points are the real users. The fake points are the rows of $\tilde{\mathbf{R}}$. The fake user ids are `agent_1..agent_N`, and the injected rows come from `cli.py::_extract_attack_interactions`. Each point is one (x, y) , its t-SNE coordinate. The call `ax.scatter` plots blue circles for the clean users and red stars for the fake ones, inside `plot_panel`. One note on the current state. The script simulates the two clouds with Gaussian generators such as `fakes_agas` and `fakes_poisonrec`, so it renders without a run. To make it data-driven, feed the real 2-D coordinates in place of those generators.

28.5 Detector: the table and the Rec@10 bars (RQ3)

Detection table `detection_mf.tex`

The loop variable `swept` is the pair (dataset, method). Run `bashbash/run_stealth_and_detect.sh`. It first produces a poisoned trace, then scores that trace with `agas.cli.evaluate-detectors-detectorsBaseDetect,DHAGCN,PCASelectUsers,GAGE,MD-CBA`. Each detector returns four numbers, which are Accuracy, Recall, Precision, and F1, on the fake-versus-real labels. Those four numbers become the four sub-columns of one block, for each pair of a detector and a method. Lower numbers mean the attack is harder to detect.

Benign Rec@10 bars (`detect_fig_defense_k5.py`)

This figure has three victims, which are NeuMF, LightGCN, and LightCCF. Each method gets one horizontal bar. The solid part of the bar is the Rec@10 after the attack. The hatched remainder is the no-attack ceiling. The printed percentage is the drop in utility.

There are two numbers per bar. Call them A and B . The value B is the Rec@ K of the clean victim, the one with no injected users. The value A is the Rec@ K of the poisoned victim. Rec@ K is (5). To get it, evaluate each test user’s held-out item on the retrained model, using the same `rank_item` machinery but scoring the user’s own item instead of the target. So B uses the clean fit and A uses the poisoned fit. These are exactly the clean and attacked pair that the transfer command already builds, namely `clean_model` and `attacked_model` in `cli.py` lines 1305–1316.

Now the drawing. In the script, `NO_ATTACK[victim]` is B and `AFTER_ATTACK[victim][method]` is A . Take the method at row y_i . The call `ax.barh(y,before)` draws the full hatched bar, whose length is B . Then `ax.barh(y,after)` draws the solid bar on top, whose length is A . The leftover gap $B - A$ is the grey segment. Finally `ax.text(...,f"{100*(B-A)/B:.0f}%")` prints the drop, all inside `draw_panel`. So the x position is the Rec@10 value, the y position is the method index, the solid bar length is A , and the ceiling is B .

28.6 Ablation: two heatmaps and a table (RQ4)

First, a word on notation. In these figures “w/o” just means “without”. Each “w/o X” row is one run of AGAS with component X switched off. We compare that run to the “Full” run, which has everything on. If the number drops a lot, then X mattered.

The full command for every row

Every ablation row is a real flag on `scripts/run_agas.py`. Nothing here needs a source edit. This one block runs the entire RQ4 sweep, all rows, across the 5 seeds. Copy, paste, run.

```
DATASET=ml-100k ; VICTIM=lightgcn
COMMON="--dataset $DATASET --victim $VICTIM --rounds 18 --n_workers 8 --budget 0.01"
mkdir -p outputs/rq4

for seed in 42 43 44 45 46; do
  # Full system (everything on) -- the baseline the "w/o" rows are compared to
  python scripts/run_agas.py $COMMON --seed $seed \
    --coordinator-agent-memory --profile-validator \
    --out outputs/rq4/full__seed${seed}.json

  # ----- component / size heatmap rows -----
  python scripts/run_agas.py $COMMON --seed $seed --random-coordinator --out outputs/rq4/
    wo_coordinator__seed${seed}.json
```

```

python scripts/run_agas.py $COMMON --seed $seed --disable-roles pr      --out outputs/rq4/
  wo_profiler__seed${seed}.json
python scripts/run_agas.py $COMMON --seed $seed --disable-roles sn      --out outputs/rq4/
  wo_sniper__seed${seed}.json
python scripts/run_agas.py $COMMON --seed $seed --disable-roles ca      --out outputs/rq4/
  wo_camouflageur__seed${seed}.json
python scripts/run_agas.py $COMMON --seed $seed --disable-roles in      --out outputs/rq4/
  wo_inactive__seed${seed}.json
python scripts/run_agas.py $COMMON --seed $seed --no-coordinator-agent-memory --out outputs/rq4/
  wo_memory__seed${seed}.json
python scripts/run_agas.py $COMMON --seed $seed --disable-signals      --out outputs/rq4/
  wo_signals__seed${seed}.json
python scripts/run_agas.py $COMMON --seed $seed --no-profile-validator  --out outputs/rq4/
  wo_validator__seed${seed}.json

# ----- strategy heatmap rows -----
for s in s1 s2 s3 s4 s5 s6 s7 s8; do
  python scripts/run_agas.py $COMMON --seed $seed --disable-strategies $s \
    --out outputs/rq4/wo_${s}__seed${seed}.json
done
done

# turn the 5-seed runs into one mean +/- 95% CI per row
python scripts/aggregate_results.py outputs/rq4 --k 10 --csv rq4.csv

```

The wrapper `bashbash/run_ablation.sh` runs exactly this (set `SEEDS="42"` for a quick one-seed smoke). To read one row on its own, just take its line. For example, the exact command for “w/o S7” is

```

python scripts/run_agas.py --dataset ml-100k --victim lightgcn \
  --rounds 18 --n_workers 8 --budget 0.01 --seed 42 \
  --disable-strategies s7 --out outputs/rq4/wo_s7__seed42.json

```

and the exact command for “w/o Profiler” is the same line with `--disable-rolespr` in place of `--disable-strategiess7`.

What each row removes (plain words)

Row	Flag	What it takes away
w/o Coordinator	<code>--random-coordinator</code>	roles are chosen at random, not by the LLM
w/o Profiler	<code>--disable-rolespr</code>	no probing or bridge discovery (those workers become camoufleurs)
w/o Sniper	<code>--disable-rolesn</code>	no targeted payload (snipers become camoufleurs)
w/o Camoufleur	<code>--disable-rolesca</code>	no benign filler cover (camoufleurs go inactive)
w/o Inactive	<code>--disable-rolesin</code>	no cool-down, everyone always acts
w/o Memory	<code>--no-coordinator-agent-memory</code>	the Coordinator forgets earlier rounds
w/o Signals	<code>--disable-signals</code>	trust, risk, validator, suppression, and alerts are hidden (only raw rank remains)
w/o Validator	<code>--no-profile-validator</code>	the profile-suspicion guardrail is off
w/o S1	<code>--disable-strategiess1</code>	no Victim Probe (skip the probe phase)
w/o S2	<code>--disable-strategiess2</code>	no Bridge Building
w/o S3	<code>--disable-strategiess3</code>	no Warm-up (prompt-level)
w/o S4	<code>--disable-strategiess4</code>	no First Push (prompt-level)
w/o S5	<code>--disable-strategiess5</code>	no Silent Slowdown (prompt-level)
w/o S6	<code>--disable-strategiess6</code>	no Profile Cleanup guardrail
w/o S7	<code>--disable-strategiess7</code>	no Safe Replacement (no suspicion lockout)
w/o S8	<code>--disable-strategiess8</code>	no Main Attack push (prompt-level)

Two honest notes. First, you can combine flags, for example `--disable-rolespr,sn` or `--disable-strategiess1,s7`. Second, S1, S2, S6, S7 and every component row are hard switches that always take effect. S3, S4, S5, S8 are behaviours the language model produces on its own, so “w/o” for those is done by telling the Coordinator in its prompt not to use them. The model usually obeys but is not forced.

Strategy heatmap (ablation_strategy_heatmap.py)

This figure has four panels, one per dataset. In each panel the rows are Full, then w/o S1 through w/o S8. The columns are three victims, which are NeuMF, LightGCN, and LightCCF. The colour of a cell, and the number in it, is HR@10. You produce the rows with the `--disable-strategiess1..s8` runs above.

Now the drawing. The script holds a matrix `VALUES[dataset]` of shape 9×3 . Row i is the ablation config, and column j is the victim. The call `ax.imshow(values, cmap="YlOrRd_r")` colours cell (i, j) by its HR@10, and `ax.text(j, i, f"{val:.1f}")` prints the number, both inside `draw_panel`. So the cell at row i , column j is the HR@10, scaled by 10^3 . Darker or brighter follows the colour bar. To fill it with your own runs, replace the `VALUES` block with the aggregate from `aggregate_results.py`.

Component and size heatmap (ablation_size_heatmap.py)

The drawing is identical to the strategy heatmap. What changes is the axes. The rows are now Full, w/o Coordinator, w/o Profiler, w/o Sniper, w/o CA, w/o IN, w/o Memory, and w/o Signals, which are the component runs above. The columns are the backbone size, which is Large, Medium, or Small. You set the size by adding `--worker-llm-model` to any of those commands. For example, one Large cell for the “w/o Sniper” row is:

```
python scripts/run_agas.py --dataset ml-100k --victim lightgcn \
```

```
--rounds 18 --n_workers 8 --budget 0.01 --seed 42 \
--disable-roles sn --worker-llm-model llama-3.3-70b \
--out outputs/rq4/wo_sniper__large__seed42.json
```

Use a 14B model for Medium and a 2B model for Small. So one cell is: run AGAS with this size and this component removed, then read the aggregated HR@10.

Backbone table ablation_backbones.tex

This uses the same size sweep over language-model backbones, grouped as frontier, large, medium, and small. You set the backbone with `--worker-llm-model` (and `--llm-model` for the Coordinator). Each cell is HR@10, NDCG@10, and Rec@50, aggregated per pair of a backbone and a method.

28.7 Efficiency: rounds, time, tokens, and budget (RQ5)

Rounds trade-off (efficiency_rounds_tradeoff.py)

This is a plot with two y-axes. The bars are HR@10 per round, on the left axis. The red line is the cumulative minutes per round, on the right axis. A vertical dashed line marks the chosen budget, $T = 18$.

The loop variable swept is the round index t . Inside one run, the per-round rank is already in `history[t].feedback.target_rank`. Convert each rank with `_rank_to_hr_ndcg` to get HR@10 at round t , then average over seeds for the bar height and its CI. The line value is the wall-clock elapsed up to round t , which the episode now records as `cumulative_sec` in each `history` entry (and `runtime_sec` for the whole run). Read the totals with `aggregate_results.py--timing`.

Now the drawing. The arrays `HR10[t]`, `HR10_CI[t]`, and `TIME_MIN[t]` are indexed by `ROUNDS`, which runs from 1 to 30. The call `ax1.bar(ROUNDS, HR10)` places a bar at $x = t$ of height `HR10[t]`, and `ax1.errorbar(...)` adds whiskers of size `HR10_CI[t]`. The twin axis call `ax2.plot(ROUNDS, TIME_MIN)` draws the point $(t, \text{minutes}(t))$. Finally `ax1.axvline(18)` marks the budget. So a bar is the pair $(x = t, y = \overline{\text{HR@10}}(t))$ and a line point is the pair $(x = t, y = \overline{\text{minutes}}(t))$.

Method versus time (efficiency_method_time_compare.py)

This shows four datasets as x-groups, with five methods in each group. Each method gets a bar, which is HR@10 on the left axis, and a line marker, which is the total runtime in minutes on the right axis.

The loop variable swept is the pair (method, dataset). The bar height is the aggregated HR@10 for that pair, from Steps 1 through 3. The line value is the mean total minutes of those runs.

Now the drawing. The arrays `HR10[m]` and `TIME_MIN[m]` are length-4 lists over the datasets. With `x=arange(4)` and a per-method `offsets`, the call `ax1.bar(x+offset[m], HR10[m])` puts method m 's bar for dataset d at $x = d + \text{offset}(m)$, with height `HR10[m][d]`. The twin call `ax2.plot(x+offset[m], TIME_MIN[m])` draws its runtime marker at the same x . So the bar height is the effectiveness, and the marker height is the cost, read on opposite axes.

Tokens until Top-10 (efficiency_token_topk.py)

This shows grouped bars on a log y-axis. The bar is the average number of language-model tokens spent before the target first enters the Top-10. There are three agentic methods across four datasets. A side table lists the inference seconds.

Here is how to get the numbers. Walk a run’s `history` to the first round t^* where `feedback.target_rank` is at most 10. Then sum the per-call tokens up to t^* . Each report carries `trace.token_usage.total_tokens`, and the run total is `token_usage.aggregate.total_tokens` from `_collect_token_usage` (line 615). Average this over seeds. The seconds column is the matched wall-clock. The command `pythonscripts/aggregate_results.py<dir>--tokens` prints the per-run token mean that feeds this figure.

Now the drawing. The arrays `TOKENS[m]` and `TIME_S[m]` are length-4 lists. The call `ax_bar.bar(x+offset[m], TOKENS[m])` with `ax_bar.set_yscale("log")` makes a bar at $x = d + \text{offset}(m)$ whose height, on a log scale, is the token count. The call `ax_bar.text(...)` prints a label such as “55K” above it. The right-hand `ax_tab.table` lists `TIME_S` row by row, dataset first and method within. So the bar height, on a log scale, is the tokens, and the table cell is the seconds.

Budget trade-off (`efficiency_fig_agents_tradeoff.py`)

This has a 2-by-2 grid of subplots, one per dataset. In each subplot there are four lines, which are HR@10, NDCG@10, HR@50, and NDCG@50. The x-axis is the fake-user budget, from 0.5% to 3.0%.

The loop variable swept is the budget b . The script `run_efficiency.sh` loops `--budget0.0050.010.020.03`, and you can add 0.015 and 0.025. Each budget value is one x-tick. Aggregate HR and NDCG per budget.

Now the drawing. The entry `VALUES[dataset][metric]` is a length-6 list over `BUDGETS=[0.5, 1, 1.5, 2, 2.5, 3]`. For each metric, the call `ax.plot(BUDGETS, VALUES[dataset][metric])` draws the point ($x = b$, $y = \text{metric}(b)$) and connects the points, one styled line per metric. So each line is one metric as the budget grows, and the four lines share the subplot.

28.8 Tables at a glance

Table (in <code>main.pdf</code>)	Produced by (this repo)	Numbers from
<code>benchmark_unpopular.tex</code> (Tab. I, main result)	<code>bash/run_performance.sh</code> over all datasets and victims	<code>option_a</code> or <code>option_b</code> ranks turned into HR and NDCG by <code>aggregate_results.py</code>
<code>benchmark_popularity_regime.tex</code>	the same, with head and mid target items	the same aggregation on head and mid target bins
<code>detection_mf.tex</code> (detector table)	<code>bash/run_stealth_and_detect.sh</code> , then <code>evaluate-detectors</code>	detector Accuracy, Recall, Precision, and F1 per method
<code>ablation_mf.tex</code> , <code>ablation_backbones.tex</code>	<code>bash/run_ablation.sh</code> with component toggles, plus <code>--worker-llm-model</code> for the backbones	HR and NDCG per ablated config, aggregated over seeds
<code>efficiency_breakdown.tex</code>	<code>bash/run_efficiency.sh</code>	per-stage wall-clock from the run timing

Remark 6 (Illustrative versus data-driven, said plainly). Today several drawing scripts carry a hard-coded data block. These are `RAW`, `VALUES`, `HR10`, `TOKENS`, `PROFILES`, and the t-SNE generators. That block is only the last inch of Step 4. It has exactly the shape that the aggregation in Part IX prints. So making a figure data-driven means one thing. You paste the aggregated array, or read the `--csv`, in place of the hard-coded one. Nothing about the geometry changes. The engine in this repo owns every number, and the scripts in `../code/` own only the drawing.

Part XI — Everything in one place

29 Every equation, and where it lives

Quantity	Equation	File and symbol
Top- K recommendation	(1)	<code>recsys/*::recommend</code> and <code>rank_item</code>
Poisoned matrix	(2)	<code>*::append_interactions</code>
HR@ K , NDCG@ K	(3),(4)	<code>experiment.tex</code> (population)
rank to HR and NDCG	(6)	<code>cli.py::_rank_to_hr_ndcg</code>
Attack objective	(7)	<code>formulation.tex</code>
Trust τ	(8)	<code>signals.py::update_after_action</code>
Risk γ	(9),(10)	<code>signals.py::update_after_action</code> and <code>apply_inactive_decay</code>
Validator ϕ	(11)	<code>signals.py::update_validator</code>
Suppression terms	(12)	<code>signals.py::EnvSignals.update</code>
Group overlap g_t	(13)	<code>signals.py::EnvSignals.update</code>
Suspicion q_t	(14)	<code>signals.py::EnvSignals.update</code>

30 Every file that matters

File	Role
<code>scripts/run_agas.py</code>	single entry point for one episode
<code>scripts/preprocess_all.py</code>	dataset into canonical CSVs
<code>scripts/aggregate_results.py</code>	runs into aggregated numbers (added by this doc)
<code>bash/run_*.sh</code>	one sweep per research question
<code>src/agas/roles.py</code> , <code>strategies.py</code>	the 4 roles and the 8 strategies
<code>src/agas/signals.py</code>	all worker and environment signals
<code>src/agas/agents/coordinator.py</code>	strategy choice, role assignment, guardrails
<code>src/agas/agents/worker.py</code>	per-role action generation
<code>src/agas/agents/defender.py</code>	black-box signal reconstruction
<code>src/agas/simulation/episode.py</code>	the round loop (Algorithm 1)
<code>src/agas/simulation/environment.py</code>	defense filter, refit, rank, item pools, bridges
<code>src/agas/recsys/targets/*</code>	the 11 victim models
<code>src/agas/metrics/embedding_cluster.py</code>	victim-side stealth metric
<code>prompts/*</code>	Coordinator and per-role prompt templates

31 The whole pipeline in one paragraph

First preprocess the datasets into canonical CSVs. Then run `scripts/run_agas.py`, or a `bash/run_*.sh` sweep, to play out T rounds. In each round a Coordinator language model reads the trust, risk, and validator signals, and the rank, acceptance, suppression, and alert signals. It picks one of eight strategies and gives each fake user one of four roles. Deterministic guardrails keep the campaign stealthy. Each round the victim is refit and the target’s rank is read again. This produces one JSON trace per run. Next, aggregate those traces across five seeds with `scripts/aggregate_results.py` into HR@ K and NDCG@ K , reported as the mean and 95% CI and scaled by 10^3 . Finally feed those numbers to the matplotlib scripts in `../code/` to draw every figure in `main.pdf`.